10주차 과제

파이썬 머신러닝 완벽 가이드 6장 (6.1~6.6) 개념정리 및 필사

6.1 차원 축소(Dimension Reduction) 개요

차원 축소: 매우 많은 피처로 구성된 다차원 데이터 세트의 차원을 축소해 새로운 차원의 데이터 세트를 생성하는 것

cf. 대표 알고리즘: PCA, LDA, SVD, NMF

차원 축소가 필요한 이유

- 차원↑ → 데이터 포인트 간 거리가 기하급수적으로 멀어짐 + 희소(sparse)한 구조 → 적은 차원에서 학습한 모델보다 **예측 신뢰도 떨어짐**
- 피처가 많으면 피처 간 상관관계 높을 가능성↑ → 선형 회귀 같은 선형 모델에서 **다중공선성** 문제로 예측 성능 저하
- 3차원 이하로 축소하면 **시각화** 가능 → 데이터를 직관적으로 해석
- 학습 데이터 크기 감소 → 학습에 필요한 **처리 능력 감소**

피처 선택 vs 피처 추출

구분	피처 선택 (feature selection)	피처 추출 (feature extraction)
방식	불필요한 피처 제거, 주요 피처만 선택	기존 피처를 저차원의 중요 피처로 압축해 추출
결과	기존 피처 그대로 사용	기존 피처와 완전히 다른 값

- 피처 추출 = 단순 압축 X, 피처를 **함축적으로 더 잘 설명할 수 있는 다른 공간으로 매핑**
  - ex. 모의고사·내신·수능·봉사활동·대외활동·수상경력 → 학업 성취도, 커뮤니케이션 능력, 문제 해결력
  - ⇒ 기존 피처로는 인지하기 어려웠던 **잠재적인 요소(Latent Factor)** 추출

> 💡 차원 축소의 진짜 의미 = 데이터 압축이 아니라 **데이터를 잘 설명하는 잠재 요소를 추출**하는 것 (PCA, SVD, NMF)

활용 분야

- **이미지**: 픽셀 수가 너무 많음 → 잠재 특성을 피처로 도출해 차원 축소 → 과적합 영향력 ↓ → 원본보다 예측 성능↑ 가능
- **텍스트**: 문서 내 단어 구성에서 숨겨진 시맨틱(Semantic) 의미나 토픽(Topic)을 잠재 요소로 찾아냄 → SVD, NMF가 시맨틱 토픽 모델링의 기반 알고리즘

6.2 PCA(Principal Component Analysis)

PCA 개요

: 여러 변수 간 상관관계를 이용해 이를 대표하는 **주성분(Principal Component)** 을 추출해 차원을 축소하는 기법

- 정보 유실 최소화 위해 **가장 높은 분산을 가지는 데이터의 축**을 찾아 그 축으로 차원 축소 → 이 축이 주성분
    - 분산 = 데이터의 특성을 가장 잘 나타내는 것으로 간주

키·몸무게 예시 (2개 피처 → 1개 주성분)

1. 데이터 변동성이 가장 큰 방향으로 축 생성
2. 새로운 축으로 데이터 투영
3. 새로운 축 기준으로 데이터 표현

축 생성 순서

- 첫 번째 축: 가장 큰 데이터 변동성(Variance) 기반
- 두 번째 축: 첫 번째 축에 **직각인 벡터(직교 벡터)**
- 세 번째 축: 두 번째 축과 직각인 벡터 ...
- ⇒ 생성된 벡터 축에 원본 데이터 투영 → 벡터 축 개수만큼의 차원으로 축소

⇒ 원본 피처 개수에 비해 **매우 작은 주성분으로 원본 데이터의 총 변동성 대부분을 설명**하는 분석법

선형대수 관점의 PCA

> 💡 입력 데이터의 **공분산 행렬을 고유값 분해** → 구한 **고유벡터에 입력 데이터를 선형 변환**

- **고유벡터** = PCA의 주성분 벡터, 입력 데이터의 분산이 큰 방향
- **고윳값(eigenvalue)** = 고유벡터의 크기 = 입력 데이터의 분산

▼ **관련 개념**

- **선형 변환**: 특정 벡터에 행렬 A를 곱해 새로운 벡터로 변환 (= 한 공간에서 다른 공간으로 투영, 행렬을 공간으로 가정)
- **분산 vs 공분산**
  - 분산: 한 개 변수의 데이터 변동
  - 공분산: 두 변수 간의 변동 → Cov(X, Y) > 0 이면 X(키) 증가할 때 Y(몸무게)도 증가
- **공분산 행렬**: 여러 변수의 공분산을 포함하는 정방형 행렬
  - 대각선 원소 = 각 변수의 분산 / 나머지 = 변수 쌍 간 공분산

	X	Y	Z
X	**3.0**	-0.71	-0.24
Y	-0.71	**4.5**	0.28
Z	-0.24	0.28	**0.91**

- **고유벡터**: 행렬 A를 곱해도 방향은 변하지 않고 크기만 변하는 벡터 → `Ax = ax` (A: 행렬, x: 고유벡터, a: 스칼라값)
  - 정방 행렬은 최대 차원 수만큼 고유벡터 가짐 (2×2 → 2개, 3×3 → 3개)
- 공분산 행렬 = **정방행렬 + 대칭행렬($A^T = A$)**
  - 대칭행렬은 항상 고유벡터를 **직교행렬**로, 고유값을 **정방 행렬로 대각화** 가능
  - ※ 교재에 "정방행렬(Diagonal Matrix)"로 표기돼 있으나 정방행렬은 Square Matrix, Diagonal Matrix는 대각행렬

공분산 행렬의 분해

$C = P \Sigma P^T$
(P: n×n 직교행렬, Σ: n×n 정방행렬(고유값 대각), P^T: P의 전치 행렬)

$= [e_1 \dots e_n] \cdot diag(\lambda_1 \dots \lambda_n) \cdot [e_1^T \dots e_n^T]$

- e_i: i번째 고유벡터 / λ_i: i번째 고유벡터의 크기
- e_1: 가장 분산이 큰 방향 / e_2: e_1에 수직이면서 다음으로 분산이 큰 방향

PCA 수행 스텝

1. 입력 데이터 세트의 **공분산 행렬** 생성
2. 공분산 행렬의 **고유벡터와 고유값** 계산

3. 고유값이 가장 큰 순으로 **K개(PCA 변환 차수만큼) 고유벡터 추출**
4. 추출된 고유벡터를 이용해 **입력 데이터 변환**

붓꽃 데이터 PCA 변환

- 4개 속성(sepal length, sepal width, petal length, petal width) → 2개 PCA 차원으로 압축
- 원본 분포(sepal length × sepal width): setosa는 구분 쉬움, versicolor·virginica는 분류 어려움
- **PCA 적용 전 스케일링 필수!** PCA는 여러 속성값을 연산 → 속성 스케일에 영향 받음 → `StandardScaler`로 평균 0, 분산 1로 변환
- `PCA(n_components=2)` → `fit()` + `transform()` → (150, 2)

결과

- pca_component_1 축 기준 setosa 명확히 구분, versicolor·virginica 일부 겹치지만 비교적 잘 구분
- `explained_variance_ratio_`: 전체 변동성에서 개별 컴포넌트가 차지하는 비율
  - [0.7296, 0.2285] → 2개 요소로 원본 변동성의 **약 95%** 설명

RandomForest 분류 비교 (cv=3)

데이터	평균 정확도
원본 (4개 속성)	약 0.96
PCA 변환 (2개 속성)	약 0.867

→ 정확도 약 10% 하락, but 속성 개수 50% 감소 고려하면 원본 특성 상당 부분 유지

코드 필사에서 이어서~

신용카드 고객 데이터 PCA 변환

- UCI Credit Card Clients Data Set: 30,000개 레코드 × 24개 속성
- Target: `default payment next month` (다음달 연체 여부, 1: 연체 / 0: 정상납부)
  - `PAY_0` → `PAY_1`, `default payment next month` → `default`로 칼럼명 변경
- `corr()` + seaborn `heatmap`으로 상관도 확인
  - **BILL_AMT1~6**: 상관도 대부분 0.9 이상
  - **PAY_1~6**: 이보다 낮지만 역시 높음

    - ⇒ 상관도 높은 속성들은 소수의 PCA로도 변동성 수용 가능

BILL_AMT 6개 → 2개 컴포넌트

- `explained_variance_ratio_`: [0.9056, 0.0510] → 2개로 약 95% 이상, 첫 번째 축만으로 90%

RandomForest(n_estimators=300) 분류 비교 (cv=3)

데이터	평균 정확도
원본 (23개 속성)	0.8171
PCA 변환 (6개 컴포넌트)	0.7961

→ 전체 속성의 약 1/4로 1~2% 성능 저하만 발생 ⇒ PCA의 뛰어난 압축 능력

- PCA 활발한 적용 분야: **컴퓨터 비전** (얼굴 인식의 Eigen-face)

코드 필사에서 이어서~

6.3 LDA(Linear Discriminant Analysis)

LDA 개요

: 선형 판별 분석법. PCA와 유사하게 저차원 공간에 투영해 차원 축소, but **지도학습 분류에서 사용하기 쉽도록 클래스를 분별할 수 있는 기준을 최대한 유지**하면서 축소

	PCA	LDA
찾는 축	입력 데이터 **변동성이 가장 큰** 축	결정 값 **클래스를 최대한 분리**할 수 있는 축
학습 유형	비지도학습	**지도학습** (클래스 레이블 필요)
분해 대상	공분산 행렬	클래스 간 분산 & 클래스 내부 분산 행렬

- **클래스 간 분산(between-class scatter)** 은 최대한 크게
- **클래스 내부 분산(within-class scatter)** 은 최대한 작게
- ⇒ 두 분산의 비율을 최대화하는 방식으로 차원 축소

LDA 수행 스텝

1. 클래스 내부 분산 행렬(S_W)과 클래스 간 분산 행렬(S_B) 구함 → 클래스별 개별 피처의 평균 벡터(mean vector) 기반
2. 두 행렬을 고유벡터로 분해
    - $S_W^T S_B = [e_1 \dots e_n] \cdot \mathrm{diag}(\lambda_1 \dots \lambda_n) \cdot [e_1^T \dots e_n^T]$

- ※ 교재 표기는 S_W^T, 일반적인 LDA 유도에서는 역행렬 $S_W^{-1}S_B$ 사용

3. 고유값이 가장 큰 순으로 K개(LDA 변환 차수만큼) 추출
4. 추출된 고유벡터로 입력 데이터 변환

붓꽃 데이터 LDA 적용

- `LinearDiscriminantAnalysis` 클래스 사용, 먼저 StandardScaler로 스케일링
- `LinearDiscriminantAnalysis(n_components=2)`
- **주의: `fit()` 호출 시 결정값(iris.target) 입력 필요!** ← 지도학습이기 때문
- 변환 결과 (150, 2), 품종별로 잘 분리됨

코드 필사에서 이어서~

6.4 SVD(Singular Value Decomposition)

SVD 개요

: 특이값 분해. PCA와 유사한 행렬 분해 기법

- PCA: **정방행렬**만 고유벡터로 분해 가능
- SVD: 정방행렬뿐 아니라 **행과 열의 크기가 다른 행렬(m×n)** 에도 적용 가능

> 💡 $A = U \Sigma V^T$

- U, V에 속한 벡터: **특이벡터(singular vector)**, 모든 특이벡터는 서로 **직교**
- Σ: 대각행렬, 대각에 위치한 0이 아닌 값 = 행렬 A의 **특이값**

▼ 분해 형태별 차원 (A: m×n)

형태	U	Σ	V^T
Full SVD	m×m	m×n	n×n
Compact SVD (Σ의 비대각 부분 & 특이값 0인 부분 제거)	m×p	p×p	p×n
Truncated SVD	Σ의 대각원소 중 **상위 몇 개만** 추출, 대응하는 U·V 원소도 제거		

넘파이 SVD 실습 (numpy.linalg.svd)

- 4×4 랜덤 행렬 생성 (개별 로우끼리 의존성 없애기 위해 랜덤)
- `U, Sigma, Vt = svd(a)` → (4,4) (4,) (4,4)
  - Sigma는 0이 아닌 값만 **1차원**으로 반환
- 복원: `np.diag(Sigma)`로 0 포함 대칭행렬로 변환 후 `U · Sigma_mat · Vt` 내적 → 원본과 동일하게 복원

로우 간 의존성 부여 시

- `a[2] = a[0] + a[1]`, `a[3] = a[0]`
- Sigma: [2.663, 0.807, 0, 0] → 2개가 0 ⇒ 선형 독립인 로우 벡터 2개 = **행렬의 랭크(Rank) 2**
- U는 앞 2열, Sigma는 앞 2개, Vt는 앞 2행만 추출 → (4,2) (2,2) (2,4) → 원본 복원 가능

코드 필사에서 이어서~

Truncated SVD

- Σ의 특이값 중 상위 일부만 추출 → 더 작은 차원으로 분해 → **원본 정확히 복원 X, but 상당 수준 근사 가능**
- 원래 차원에 가깝게 잘라낼수록(Truncate) 원본에 더 가깝게 복원
- 사이파이에서 지원 (넘파이 X)
  - 일반 SVD: `scipy.linalg.svd`
  - Truncated SVD: `scipy.sparse.linalg.svds` (희소 행렬만 지원)
- 6×6 행렬: Normal SVD → (6,6) (6,) (6,6) / `svds(k=4)` → (6,4) (4,) (4,6) → 근사적으로 복원

코드 필사에서 이어서~

사이킷런 TruncatedSVD 클래스

- U, Sigma, Vt 행렬을 반환하지 **않음**
- PCA처럼 `fit()` + `transform()` → 원본 데이터를 K개 주요 컴포넌트로 축소
- 원본 데이터를 **U*Sigma 행렬에 선형 변환**해 생성
- 붓꽃 데이터 2개 컴포넌트 변환 → PCA처럼 품종별 클러스터링 가능할 정도로 고유성 가짐

TruncatedSVD vs PCA

- 스케일링 후 두 변환 비교 → 변환 값 차이, components_ 차이 모두 **에 가까움** ⇒ 동일한 변환
- ⇒ 데이터 중심이 동일해지면 SVD와 PCA는 동일 = **PCA가 SVD 알고리즘으로 구현됨**
- 차이: PCA는 **밀집 행렬(Dense Matrix)** 만 가능, SVD는 **희소 행렬(Sparse Matrix)** 도 가능

활용: 이미지 압축을 통한 패턴 인식, 신호 처리, 텍스트 토픽 모델링 기법 **LSA(Latent Semantic Analysis)** 의 기반 알고리즘

코드 필사에서 이어서~

6.5 NMF(Non-Negative Matrix Factorization)

NMF 개요

: Truncated SVD와 같은 **낮은 랭크를 통한 행렬 근사(Low-Rank Approximation)** 방식의 변형
→ 원본 행렬의 모든 원소 값이 **양수(0 이상)** 라는 게 보장되면, 두 개의 기반 양수 행렬로 분해

> 💡 V (4×6) ≈ W (4×2) × H (2×6)

- **W**: 길고 가는 행렬 (원본 행 크기와 같고, 열 크기보다 작음) → 원본 **행**에 대해 잠재 요소 값이 얼마나 되는지
- **H**: 작고 넓은 행렬 (원본 행 크기보다 작고, 열 크기와 같음) → 잠재 요소가 원본 **열(속성)** 로 어떻게 구성됐는지
- 행렬 분해(Matrix Factorization) = SVD 같은 행렬 분해 기법을 통칭

붓꽃 데이터 NMF 적용

- 사이킷런 `NMF(n_components=2)` → `fit()` + `transform()` → 2개 컴포넌트로 시각화

코드 필사에서 이어서~

활용

- 이미지 변환·압축, 텍스트 토픽 도출, 문서 유사도 및 클러스터링
- **추천(Recommendations)**: 사용자-평가 순위(user-Rating) 데이터를 행렬 분해 → 평가하지 않은 상품의 잠재 요소 추출 → 평가 순위 예측 → 높은 순위 상품 추천

- = **잠재 요소(Latent Factoring) 기반 추천 방식**

6.6 정리

차원 축소

- 많은 피처로 이뤄진 데이터를 더 직관적으로 이해 가능
- 단순히 피처 개수를 줄이는 것보다 **데이터를 잘 설명하는 잠재 요소 추출**에 큰 의미
- 많은 차원을 가지는 이미지, 텍스트에서 PCA, SVD 등이 활발히 사용됨

PCA

- 변동성이 가장 큰 축 → 이에 직각인 축을 차원 개수만큼 구한 뒤 투영
- 공분산 행렬 기반 고유벡터 생성 → 고유벡터에 입력 데이터 선형 변환

LDA

- PCA와 유사하지만, **클래스를 최대한 분리할 수 있는 축**을 찾음 (지도학습)

SVD & NMF

- 고차원 행렬을 두 개의 저차원 행렬로 분리하는 행렬 분해 기법
- 원본 행렬의 잠재 요소 추출 → **토픽 모델링, 추천 시스템**에서 활발히 사용

알고리즘	핵심	대상	특징
PCA	분산 최대 축	공분산 행렬 (정방)	비지도, 스케일링 필수, 밀집 행렬
LDA	클래스 분리 최대 축	클래스 간/내부 분산 행렬	지도학습, fit에 레이블 필요
SVD	$A = U\Sigma V^T$	m×n 행렬	희소 행렬 가능, LSA 기반
NMF	$V \approx W \times H$	양수 행렬	토픽 모델링, 추천